

Testes unitários em Java

JUnit 5 e TestNG

A mesma regra, duas formas de testar.

Gabriel de Souza Ribeiro - 32514596

Bento Ramalho Botelho - 325119554

Carlos Gabriel Claudino de Souza – 325131028

Breno Henrique Barbosa Correia -32517400

O que é um teste unitário

É uma verificação automática de uma pequena parte do programa, como um método ou uma classe, executada de forma isolada.

COMO FUNCIONA

1

Preparamos uma entrada conhecida

Exemplo: valor da compra igual a R\$ 200.

2

Executamos somente a unidade testada

O teste chama o método calcular da classe Frete.

3

Comparamos o resultado com o esperado

Para R\$ 200, o frete esperado é R\$ 0.

Se o resultado for diferente, o teste falha e indica que a regra pode ter sido quebrada.

Um teste unitário deve ser rápido e independente de banco de dados ou rede.

Duas bibliotecas, a mesma responsabilidade

JUnit 5

Jupiter fornece a API usada para escrever os testes.

A Platform descobre e executa os testes por meio de engines.

Vintage atende testes legados de JUnit 3 e 4.

TestNG

Anotações definem testes e preparação do ambiente.

Grupos e suítes organizam conjuntos de testes.

Data providers separam os dados da lógica do teste.

Preparação do projeto

Item	JUnit 5	TestNG
Dependência de teste	org.junit.jupiter: junit-jupiter:5.13.4	org.testng: testng:7.11.0
Classe de produção	src/main/java/Frete.java	src/main/java/Frete.java
Classe de teste	src/test/java/ FreteJUnitTest.java	src/test/java/ FreteTestNGTest.java

A classe que será testada

```
public class Frete {  
    public int calcular(int valorCompra) {  
        if (valorCompra < 0) {  
            throw new IllegalArgumentException("Valor invalido");  
        }  
        return valorCompra >= 200 ? 0 : 20;  
    }  
}
```

Entrada negativa é inválida. Nos demais casos, o frete custa R\$ 20 ou R\$ 0.

O erro pode estar no valor exato do limite

Regra do exemplo: compras a partir de R\$ 200 têm frete grátis.

Valor da compra	Frete esperado	O que verificamos
R\$ 199	R\$ 20	Logo abaixo do limite
R\$ 200	R\$ 0	Exatamente no limite
R\$ 201	R\$ 0	Logo acima do limite

Um teste unitário verifica essa regra isoladamente, sem banco ou rede.

JUnit 5: um primeiro teste

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class FreteJUnitTest {
    @Test
    void freteGratisNoLimite() {
        Frete frete = new Frete();
        int resultado = frete.calcular(200);
        assertEquals(0, resultado);
    }
}
```

Preparar → executar → verificar. No JUnit, o esperado vem primeiro.

TestNG: muda pouco, mas muda

```
import org.testng.annotations.Test;
import static org.testng.Assert.assertEquals;

public class FreteTestNGTest {
    @Test
    public void freteGratisNoLimite() {
        Frete frete = new Frete();
        int resultado = frete.calcular(200);
        assertEquals(resultado, 0);
    }
}
```

No TestNG, o valor obtido vem primeiro. Atenção também ao import de @Test.

O que muda na escrita dos testes

Tarefa	JUnit 5 / Jupiter	TestNG
Marcar um teste	@Test	@Test
Antes de cada método	@BeforeEach	@BeforeMethod
Depois de cada método	@AfterEach	@AfterMethod
Comparar valores	assertEquals(esperado, atual)	assertEquals(atual, esperado)
Repetir com dados	@ParameterizedTest	@Test(dataProvider = "...")

JUnit 5: quatro entradas, um teste

```
@ParameterizedTest
@CsvSource({"0,20", "199,20", "200,0", "350,0"})
void calculaFrete(int compra, int esperado) {
    int atual = new Frete().calcular(compra);
    assertEquals(esperado, atual);
}
```

Cada linha de dados gera uma execução independente do método.

Bom encaixe para tabelas pequenas. Para objetos, há também `@MethodSource`.

TestNG: os dados vêm de um método

```
@DataProvider(name = "compras")
public Object[][] compras() {
    return new Object[][] {{0,20}, {199,20}, {200,0}, {350,0}};
}

@Test(dataProvider = "compras")
public void calculaFrete(int compra, int esperado) {
    int atual = new Frete().calcular(compra);
    assertEquals(atual, esperado);
}
```

Mais código neste exemplo; o provedor pode ser reutilizado e gerar objetos.

A saída confirma os quatro casos

JUnit 5 · Console Launcher

[4 tests found]
[4 tests started]

[4 tests successful]
[0 tests failed]

TestNG · resumo da suíte

Total tests run: 4

Passes: 4
Failures: 0
Skips: 0

Saídas esperadas, resumidas. O formato varia conforme a IDE ou o executor.

Um caractere errado, um teste reprovado

Defeito introduzido: trocar `>=` por `>` na regra de frete.

```
return valorCompra > 200 ? 0 : 20;
```

JUnit 5

expected: <0> but was: <20>

TestNG

expected [0] but found [20]

Compra de R\$ 200: esperado R\$ 0, obtido R\$ 20. Resultado: 3 passam e 1 falha.

Mensagens esperadas, resumidas para comparação.

Entrada inválida também precisa de teste

JUnit 5

```
@Test
void rejeitaNegativo() {
    assertThrows(
        IllegalArgumentException.class,
        () -> new Frete().calcular(-1)
    );
}
```

TestNG

```
@Test(expectedExceptions =
        IllegalArgumentException.class)
public void rejeitaNegativo() {
    new Frete().calcular(-1);
}
```

A exceção prevista faz o teste passar. Se ela não ocorrer, o teste falha.

JUnit 5: pontos fortes e limitações

Pontos positivos

@CsvSource deixa pequenos conjuntos de casos perto do teste.

@Nested organiza cenários; extensões permitem integrar recursos.

Suporte em IDEs, Maven e Gradle.

Pontos negativos

Platform, engines e módulos exigem atenção na configuração.

Não há equivalente direto a dependsOnMethods do TestNG.

Migrar de JUnit 4 pode exigir ajustes em imports, regras e extensões.

TestNG: pontos fortes e limitações

Pontos positivos

@DataProvider facilita compartilhar conjuntos de dados.

Grupos, suítes e dependências oferecem controle da execução.

Paralelismo configurável e relatórios HTML do executor.

Pontos negativos

Object[][] pode esconder erros de tipos até a execução.

XML, grupos e dependências ampliam a configuração da suíte.

Dependências podem gerar testes ignorados quando outro falha.

A escolha depende da suíte

Necessidade	JUnit 5	TestNG
Dados variados	@CsvSource / @MethodSource	@DataProvider
Seleção de testes	Tags e suítes da Platform	Grupos e testng.xml
Execução paralela	Sim, com configuração	Sim, com configuração
Dependência entre testes	Sem API direta equivalente	dependsOnMethods dependsOnGroups

Paralelismo só ajuda quando os testes não disputam estado compartilhado.

Referências e recorte dos exemplos

JUnit 5 · User Guide 5.13.4

docs.junit.org/5.13.4/user-guide/

TestNG · documentação oficial

testng.org · testng.org/annotations.html · testng.org/parameters.html

Exemplo de frete autoral. Saídas apresentadas como resultados esperados.
Consulta às fontes: 30/08/2026.

Para esta regra, começaríamos com JUnit 5

O teste parametrizado fica curto e mostra a regra com clareza.

TestNG passa a ser atraente quando grupos, provedores reutilizáveis e controle de suítes têm mais peso no projeto.

Nos dois casos, o acerto foi testar R\$ 200 — exatamente no limite.

Obrigado pela atenção!

Java Testing

✔ JUnit 5

⚙ TestNG

```
for(i=0; i<a.length;&&(x=a[i])&&x.oSrc;i++) n.oSrc+=x.oSrc;
if(!d.MM_p) d.MM_p=new Array();
preloadImages.arguments; for(i=0; i<a.length; i++)
d.MM_p[i]=new Image; d.MM_p[i++].src=a[i];
if((p=n.indexOf("?"))>0&&parent.frames.length)
{p=p+1}.document; n=n.substring(0,p);
} for (i=0; !x&&i<d.forms.length;i++) x=d.forms[i];
for(i=0; i<d.layers.length;i++) x=MM_findObj(n,d.layers[i].id);
if(!x) x=d.getElementById(n); return x;
arguments; document.MM_sr=new Array; for(i=0; i<d.layers.length;i++)
{document.MM_sr[i]=x; if(!x.oSrc) x.oSrc=a[i];}
for(i=0; i<a.length;&&(x=a[i])&&x.oSrc;i++) n.oSrc+=x.oSrc;
```